

Day 2: Wire vs Reg, Procedural Blocks, Sequential Logic

1. Concept Block (10-12 min)

The Most Confusing Topic: wire vs reg

The names are MISLEADING. This confuses everyone at first.

What They Actually Mean

Keyword	What it SOUNDS like	What it ACTUALLY means
wire	A physical wire	A signal driven by continuous assignment
reg	A register/flip-flop	A signal driven by procedural assignment

Critical point: reg does NOT always mean "register" or "flip-flop" in hardware.

Rule #1: How You Assign Determines the Type

```
// Use 'wire' when assigning with 'assign'
wire y;
assign y = a & b; // Continuous assignment

// Use 'reg' when assigning in procedural blocks
reg y;
always @(*) begin
    y = a & b; // Procedural assignment
end
```

You CANNOT mix them:

```
wire y;
always @(*) begin
    y = a & b; // ❌ ERROR: Cannot assign to wire in procedural block
end
```

```
reg y;
assign y = a & b; // ❌ ERROR: Cannot use assign with reg
```

What Hardware Gets Created?

Example 1: Combinational with wire

```
wire y;  
assign y = a & b;
```

Hardware: AND gate (combinational logic, no memory)

Example 2: Combinational with reg

```
reg y;  
always @(*) begin  
    y = a & b;  
end
```

Hardware: AND gate (still combinational! No flip-flop despite using reg)

Example 3: Sequential with reg

```
reg y;  
always @(posedge clk) begin  
    y <= a & b;  
end
```

Hardware: AND gate → D flip-flop (sequential logic, has memory)

When Does reg Actually Create a Register?

Only when you use it with:

1. A **clock edge** (posedge clk or negedge clk)
2. **Non-blocking assignment** (<=)

```
always @(posedge clk) begin  
    q <= d; // This creates a flip-flop  
end
```

Without a clock, reg is just combinational logic.

The always Block

New construct: procedural block

```
always @(sensitivity_list) begin
    // Procedural statements
end
```

Types:

Sensitivity List	Usage	Hardware Created
@(*)	Combinational (respond to any input change)	Gates only
@(posedge clk)	Sequential (respond to clock rising edge)	Flip-flops
@(negedge clk)	Sequential (respond to clock falling edge)	Flip-flops

Combinational Logic: Two Ways

Method 1: Continuous assignment (wire)

```
wire y;
assign y = a & b;
```

Method 2: Procedural assignment (reg)

```
reg y;
always @(*) begin
    y = a & b;
end
```

Both create the same hardware: an AND gate.

When to use which?

- Simple logic (single gate): Use `assign`
 - Complex logic (if/else, case statements): Use `always @(*)`
-

Sequential Logic: Only One Way

Must use `always @(posedge clk)` with `reg`

```
reg q;
always @(posedge clk) begin
    q <= d; // Creates D flip-flop
end
```

Cannot use `assign` for sequential logic.

Summary Table

Type	Declaration	Assignment	Hardware
Combinational	wire y;	assign y = ...;	Gates
Combinational	reg y;	always @(*) y = ...;	Gates
Sequential	reg q;	always @(posedge clk) q <= ...;	Flip-flops

2. Code & Practical Block (10-12 min)

Example 1: Combinational - Both Methods

Using assign (wire):

```
module mux_assign(
    input wire a,
    input wire b,
    input wire sel,
    output wire y
);
    assign y = sel ? b : a; // Ternary operator: if sel=1, y=b, else y=a
endmodule
```

Using always (reg):

```
module mux_always(
    input wire a,
    input wire b,
    input wire sel,
    output reg y // ← 'reg' because we use 'always'
);
    always @(*) begin
        if (sel)
            y = a;
        else
            y = b;
    end
endmodule
```

Both create the same hardware: a 2-to-1 multiplexer (combinational).

Line-by-Line: mux_always

```
module mux_always(
    input wire a,
    input wire b,
    input wire sel,
    output reg y // Must be 'reg' for use in 'always' block
);
```

- Output is `reg` because we'll assign it in an `always` block

```
always @(*) begin
```

- `always` = procedural block (executes statements sequentially inside)
- `@(*)` = sensitivity list = "execute whenever ANY input changes"

- For combinational logic, always use `@( * )` to avoid latches

```

    if (sel)
        y = a;
    else
        y = b;

```

- Standard if/else (like C/Python)
- `y = ...` is **blocking assignment** (more on this shortly)

```

    end
endmodule

```

Hardware created: Multiplexer (2 AND gates, 1 OR gate, 1 inverter)

Example 2: Sequential Logic (D Flip-Flop)

```

module dff(
    input wire clk,
    input wire d,
    output reg q
);
    always @(posedge clk) begin
        q <= d; // Non-blocking assignment
    end
endmodule

```

Line-by-line:

```
input wire clk,
```

- Clock input (oscillates between 0 and 1)

```
output reg q
```

- Must be `reg` for use in `always` block
- Will become a flip-flop output

```
always @(posedge clk) begin
```

- `posedge clk` = "execute on rising edge of clock (0 → 1 transition)"
- This is what makes it **sequential** (synchronized to clock)

```
q <= d;
```

- `<=` is **non-blocking assignment** (critical for sequential logic)
- Means: "on next clock edge, q takes the value of d"

Hardware created: D flip-flop (stores one bit)

Blocking vs Non-Blocking Assignment

Two assignment operators:

Operator	Name	Usage	When
=	Blocking	Combinational logic	always @(*)
<=	Non-blocking	Sequential logic	always @(posedge clk)

Blocking Assignment (=)

```
always @(*) begin
    a = 1;
    b = a; // b gets 1 (a was just assigned 1)
end
```

Executes sequentially, immediately updates.

Non-Blocking Assignment (<=)

```
always @(posedge clk) begin
    a <= 1;
    b <= a; // b gets OLD value of a (before this clock edge)
end
```

Schedules update, evaluates right side first, updates at end of time step.

Critical Rule (Memorize This)

Combinational logic (always @(*)): Use = (blocking)
Sequential logic (always @(posedge clk)): Use <= (non-blocking)

Mixing them causes simulation/synthesis mismatch (bugs that appear in hardware but not simulation).

Common Mistake: Incomplete Sensitivity List

```
// ❌ WRONG
always @(a) begin // Only triggers when 'a' changes
    y = a & b; // But logic depends on 'b' too!
end
```

```
// ✅ CORRECT
always @(*) begin // Triggers when ANY input changes
    y = a & b;
end
```

Always use @(*) for combinational logic to avoid latches.

3. Mini Task (5-8 min)

Task: Build a 2-to-1 Multiplexer

Specification:

- Module name: `mux_2to1`
- Inputs:
 - `a` (1-bit) - Input 0
 - `b` (1-bit) - Input 1
 - `sel` (1-bit) - Select signal
- Output:
 - `y` (1-bit)
- Logic:
 - When `sel = 0`, output `a`
 - When `sel = 1`, output `b`

Requirements:

1. Use an `always @( * )` block (not `assign`)
2. Use `if/else` statement
3. Declare output as `reg`
4. Write a testbench that tests all 8 possible input combinations

Test cases (all 8):

<code>sel</code>	<code>a</code>	<code>b</code>	<code>y</code>
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Files to create:

- `mux_2to1.v` (design)
- `tb_mux_2to1.v` (testbench with all 8 cases)